

LAB EXPERIMENT ANALYSIS REPORT

Experiment 3: Implementation of Simple Client-Server File Transfer Protocol Using Java and Cisco Packet Tracer

Student Name: Shaik Muzzammil

Roll Number: CH.SC.U4CSE24041

Section: CSE-A

Course: 23CSE302 - Computer Networks

Institution: Amrita Vishwa Vidyapeetham, Chennai Campus

Date of Experiment: 29/06/2026

Date of Submission: 02/07/2026

Maximum Marks: 10

TABLE OF CONTENTS

1. Executive Summary
2. Experiment Objectives
3. Technical Overview
 - 3.1 File Transfer Protocol (FTP) Fundamentals
 - 3.2 Java Socket Programming for File I/O
 - 3.3 Cisco Packet Tracer Network Simulation
4. Part A: Java Implementation Analysis
 - 4.1 Server-Side Implementation
 - 4.2 Client-Side Implementation
 - 4.3 Code Explanation & Walkthrough
5. Part B: Cisco Packet Tracer Implementation
 - 5.1 Network Topology Design
 - 5.2 FTP Server Configuration
 - 5.3 Client Configuration & Testing
6. Screenshot Analysis
7. Results & Inference
8. Lab Scenario Solutions
9. Viva Questions & Technical Discussion
10. Key Learnings & Takeaways

1. EXECUTIVE SUMMARY

This experiment provides comprehensive hands-on experience with File Transfer Protocol (FTP) implementation using two distinct approaches: Java socket programming for custom file transfer application development, and Cisco Packet Tracer network simulation for understanding FTP in realistic network environments. The dual-track approach enables students to understand both low-level protocol implementation details and high-level network infrastructure configuration. The Java component demonstrates practical socket-based file transfer where a server application reads local files and transmits data to connected clients over TCP sockets. This implementation covers essential concepts including `ServerSocket` creation, `FileInputStream/OutputStream` usage, buffer-based streaming for efficiency, and proper resource cleanup patterns. The code showcases production-quality error handling with file existence validation and exception management throughout the transfer process. The Cisco Packet Tracer component extends learning into network infrastructure design by requiring students to build complete network topologies including routers, switches, servers, and client workstations. Students configure IP addressing across multiple subnets, implement routing between network segments, configure FTP services with user authentication and permission management, and validate connectivity through ping testing before performing actual file operations. Key achievements include: successful bidirectional file transfer via custom Java application, working FTP server configuration in simulated environment, proper network address planning across LAN/WAN boundaries, and understanding of FTP command-line interface for upload/download operations. This experiment establishes foundational knowledge applicable to enterprise file sharing systems, content delivery networks, and secure data exchange mechanisms.

2. EXPERIMENT OBJECTIVES

Primary Learning Outcomes: • Gain exposure to FTP (File Transfer Protocol) concepts and implementation techniques • Understand how file transfer can be simulated between client and FTP server applications • Develop proficiency in Java socket programming for network file I/O operations • Learn Cisco Packet Tracer tool for network simulation and protocol analysis **Secondary Learning Outcomes:** • Master TCP socket-based file streaming with buffer optimization • Configure multi-device network topologies with proper IP addressing • Implement FTP service configuration with authentication and access control • Perform network troubleshooting using connectivity testing tools • Compare custom protocol implementation with standard FTP protocol behavior

3. technical Overview

3.1 File Transfer Protocol (FTP) Fundamentals

The File Transfer Protocol represents one of the oldest and most widely-used application-layer protocols in TCP/IP networking. Designed for reliable file distribution across heterogeneous systems, FTP operates in client-server mode where the client initiates all transfers and the server responds to requests. This experiment implements both a simplified custom version and standard FTP using industry tools. **FTP Architecture Model:** FTP uses two separate TCP connections for control and data transfer: the control channel handles commands (USER, PASS, LIST, RETR, etc.) and authentication, while the data channel carries actual file content. Our Java implementation simplifies this by using a single connection for both control and data, which is acceptable for basic file transfer but would cause issues with large files or concurrent operations in production environments. **Transfer Modes:** FTP supports multiple transfer modes: ASCII mode for text files with format conversion between different operating systems, and Binary (IMAGE) mode for raw byte-level transfer preserving exact file contents. Our implementation uses binary mode exclusively since we handle arbitrary file types without interpretation requirements. **Active vs Passive Mode:** Standard FTP supports Active mode (client opens data port for server connections back) and Passive mode (server specifies port for client to connect). Our simplified implementation uses only active mode where the server pushes data to connecting clients, matching the push model common in modern messaging architectures.

3.2 Java Socket Programming for File I/O

Java's `java.io` and `java.net` packages provide comprehensive abstractions for network programming and file handling that our implementation leverages for building the file transfer system. Understanding these APIs is crucial for developing robust network applications. **ServerSocket Class:** Creates the server-side listening endpoint bound to a specific port. Key methods include: `bind()` to associate with local address and port; `accept()` to block until client connection arrives returning dedicated `Socket` object; `close()` to release resources. Our implementation uses port 5000 as the well-known service port, avoiding conflicts with system services while remaining in the user-space port range (above 1024). **Socket Class:** Represents an established connection between two endpoints. Provides `InputStream` and `OutputStream` for bidirectional data flow. Our client creates `Socket` objects via `new Socket(serverAddress, port)` which performs TCP three-way handshake transparently. The resulting streams enable file data reception through `read()` operations and transmission via `write()` calls. **FileInputStream / FileOutputStream:** These classes bridge between file system storage and network sockets. `FileInputStream` reads local file data into memory buffers (`byte[]` arrays), while `FileOutputStream` writes received network data to persistent storage. Both use buffered I/O internally for performance, though our explicit 4096-byte buffer provides additional

control over chunk sizes during transfer. **Error Handling Patterns:** Network I/O throws checked exceptions including `IOException` (connection failures, reset connections), `FileNotFoundException` (missing source file), and `SocketException` (protocol errors). Our implementation catches these at appropriate points: file existence check before transfer initiation prevents attempting sends of non-existent resources; try-catch blocks around socket operations enable graceful failure reporting rather than application crashes.

3.3 Cisco Packet Tracer Network Simulation

Cisco Packet Tracer provides visual network simulation environment widely used in networking education and certification preparation. For this experiment, it serves as platform for implementing standard FTP protocol without writing low-level protocol code, allowing focus on network architecture and configuration aspects. **Device Categories Used:** End Devices: PC-PT (workstation simulation), Server-PT (FTP service hosting). Network Devices: Router 2911 (Layer 3 routing between subnets), Switch 2950-24 (Layer 2 local segment switching). Connections: Copper Straight-Through (PC to Switch, Switch to Router), Console Cable (PC to Router for configuration). **Addressing Scheme Design:** Our topology implements two-subnet architecture separated by router: LAN segment (192.168.1.0/24) containing Faculty PC (.2) and Student PC (.3); WAN segment (10.0.0.0/8) containing Amritapuri File Server (.2). Router interfaces: G0/0 (LAN side, 192.168.1.1) and G0/1 (WAN side, 10.0.0.1). This design demonstrates inter-VLAN routing concepts and real-world network segmentation practices. **FTP Service Configuration:** Cisco's Server-PT module includes built-in FTP service accessible via Services tab. Configuration involves: enabling FTP service toggle, creating user accounts with username/password credentials, assigning permissions (Read, Write, Load - RWL allows full file operations), and specifying root directory for user's file access scope. We configured user "Amrita" with password "amma@123" having RWL permissions on default directory.

4. PART A: JAVA IMPLEMENTATION ANALYSIS

4.1 Server-Side Implementation (FileServer.java)

The FileServer.java implementation demonstrates a single-threaded, single-client file transfer server following clean separation of concerns between network setup, file operations, and resource management phases. **Class Structure & Entry Point:** Public static void main(String[] args) serves as entry point, following Java convention. Hardcoded constants define PORT (5000) and fileName ("sample.txt") for simplicity; production implementations would externalize these to command-line arguments or configuration files. The method wraps entire server lifecycle in try-catch for centralized error handling. **Server Initialization Sequence:** 1. ServerSocket(port) creates listening socket bound to specified port on all interfaces (0.0.0.0 means accept connections from any local network interface). 2. System.out.println() outputs status messages indicating server readiness and waiting state. 3. socket.accept() blocks until client connects, returning dedicated Socket object for this session. **File Validation Logic:** Before transferring, the code checks File.exists(fileName) to verify requested file exists locally. If missing, prints error message, closes both sockets gracefully, and returns early—preventing null pointer exceptions during subsequent FileInputStream construction. This defensive programming pattern prevents confusing runtime errors. **Transfer Execution Loop:** 1. FileInputStream(file) opens source file for reading, positioning at beginning. 2. byte[4096] buffer array allocated—4KB chunks balance memory efficiency against I/O call overhead. 3. while ((bytesRead = fis.read(buffer)) != -1) loop reads until EOF, writing each chunk to os.write(buffer, 0, bytesRead). 4. os.flush() ensures all buffered data transmitted before socket closure. **Cleanup Phase:** fis.close() releases file handle; socket.close() ends client session; serverSocket.close() stops listening. Order matters: output stream before input stream, client before server to prevent data loss from buffer flush issues.

4.2 Client-Side Implementation (FileClient.java)

The FileClient.java implementation acts as file receiver, initiating connections to server and persisting received data to local storage. It demonstrates client-side connection patterns applicable to any TCP client application. **Connection Establishment:** 1. Socket(serverAddress, port) creates socket and initiates TCP connection to specified host (127.0.0.1 for localhost testing). Connection attempt occurs synchronously; in production, timeout configuration would prevent indefinite blocking on unreachable hosts. 2. Successful connect() returns connected socket ready for data reception. **Reception and Storage Loop:** 1. socket.getInputStream() obtains input stream for reading incoming data. 2. FileOutputStream(outputFile) creates local file for writing received content. 3. Identical 4096-byte buffer loop reads from input stream until EOF (server closes connection or sends all data). 4. fos.write(buffer, 0, bytesRead) persists each chunk to disk. **Completion Reporting:** System.out.println() confirms successful receipt and displays saved filename for verification. In production, return codes or thrown exceptions would indicate

success/failure to calling code. **Resource Cleanup:** `fos.close()` flushes remaining data to disk; `socket.close()` terminates network connection. Finally block ensures cleanup even if errors occur mid-transfer.

5. PART B: CISCO PACKET TRACER IMPLEMENTATION

5.1 Network Topology Design

The network topology implements a hierarchical three-tier architecture representing typical enterprise network segmentation: **Access Layer (Layer 2):** Switch 2950-24 provides connectivity for end devices within same broadcast domain. Two PCs (Faculty PC: 192.168.1.2, Student PC: 192.168.1.3) connect via copper straight-through cables, enabling peer communication within LAN segment. Switch learns MAC addresses and forwards frames based on its MAC address table. **Distribution Layer (Layer 3):** Router 2911 interconnects LAN and WAN segments, performing IP-level forwarding. GigabitEthernet0/0 (LAN interface: 192.168.1.1/24) connects to switch; GigabitEthernet0/1 (WAN interface: 10.0.0.1/8) connects toward server. Router maintains separate routing tables for each interface, enabling cross-subnet packet delivery. **Server Placement:** Amritapuri File Server resides in WAN segment (10.0.0.2) behind router, simulating DMZ placement common in enterprise networks. This positioning requires clients to traverse router for access, demonstrating real-world security boundary concepts even in lab environment. **Console Connection:** Additional console cable from Faculty PC to Router provides out-of-band management access for device configuration, mirroring real-world data center practices where administrative access differs from data path.

5.2 FTP Server Configuration

FTP server configuration follows standard procedure for setting up authenticated file services: **Service Activation:** Navigate to Server-PT > Services tab > FTP section. Toggle service state to ON. This starts FTP daemon process listening on default port 21 (though internal Packet Tracer may use alternate porting for simulation purposes). **User Account Creation:** User Setup section enables adding authentication credentials. We created user "Amrita" with password "amma@123"—demonstrating username/password authentication that FTP requires. In production, strong passwords and account lockout policies would apply. **Permission Assignment:** Permissions checkboxes determine user capabilities: Read: View directory listings and download files Write: Upload new files to server Load: Execute scripts or programs (not needed for basic FTP) RWL combination grants full file management appropriate for trusted internal network environments. **Root Directory Specification:** Default directory determines where uploaded files appear in server filesystem. Left unspecified here, defaults to user's home directory or /ftp/ root depending on server implementation.

5.3 Client Configuration & Testing

Client PCs perform FTP operations using command-line interface available in Desktop > Command Prompt: **Connection Initiation:** ftp 10.0.0.2 command initiates FTP connection to server at WAN address. System prompts for username ("Amrita") and password

("amma@123"). Successful authentication displays "230 Login successful" message and ftp> prompt indicates readiness for commands. **Directory Listing:** dir command (or ls in some variants) lists current remote directory contents, confirming file availability and permissions. Verifies msg.txt presence after upload. **Upload Operation (Faculty PC):** put msg.txt command uploads local file to server's current directory. Shows transfer progress indicators (hash marks, percentage for large files) and completion confirmation. Enables server-to-client file distribution scenario. **Download Operation (Student PC):** get msg.txt command retrieves file from server to local machine's current directory. Demonstrates client-initiated pull model common in content distribution. **Session Termination:** quit command gracefully ends FTP session, releasing server resources. dir command on local machine verifies downloaded file existence.

6. SCREENSHOT ANALYSIS

The original lab document contains 25 pages comprehensively documenting the complete experiment including source code, execution evidence, and results validation. Screenshots are organized by category: **Documentation Pages (1-8)**: Cover page with official branding and student identification. Experiment objectives stating FTP exposure goals. Complete FileServer.java and FileClient.java source code listings showing implementation details. Output screenshot showing successful file transfer execution. **Network Topology Diagrams (13)**: Visual representation of complete network architecture including all devices, connection types (copper straight-through, console), and IP address labels at each interface. Essential for understanding physical and logical layout relationships. **Connectivity Test Results (14-16)**: Command prompt captures showing ping test results: Faculty PC to Student PC, Faculty PC to Router LAN interface, Faculty PC to Router WAN interface, Faculty PC to File Server. All tests should show 100% packet delivery with 0% loss, confirming proper routing configuration. **FTP Session Captures (17-18)**: Terminal screenshots showing actual FTP command sequences: connection establishment with credential prompts, directory listing display, put/get command execution with progress feedback, and quit sequence. These provide undeniable evidence of successful protocol implementation. **Lab Scenario Solutions (21-25)**: Detailed answers to technical questions about file size, error handling, IP configuration, screenshot demonstration requirements, and network troubleshooting challenges. Includes viva preparation questions covering socket programming theory, FTP limitations, and Cisco Packet Tracer methodology.

Key Screenshots from Lab Document:

AMRITA VISHWA VIDYAPEETHAM – CHENNAI CAMPUS		
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING		
23CSE302 – COMPUTER NETWORKS		
EXPERIMENT 3		
IMPLEMENTATION OF SIMPLE CLIENT- SERVER FILE TRANSFER PROTOCOL USING JAVA AND CISCO PACKET TRACER		
Name: Shaik Muzzammil		Max Marks (10)
Roll No.: CH.SC.U4CSE24041	Section: CSE-A	
Date of Exp: 29/06/2026	Date of Submission: 02/07/2026	
OBJECTIVES:		
1. Students will have exposure to FTP using Java		
2. Students will have an understanding how file transfer can be simulated between client and FTP server in Cisco Packet Tracer using FTP protocol		

Figure: Cover Page



Figure: Output Screenshot

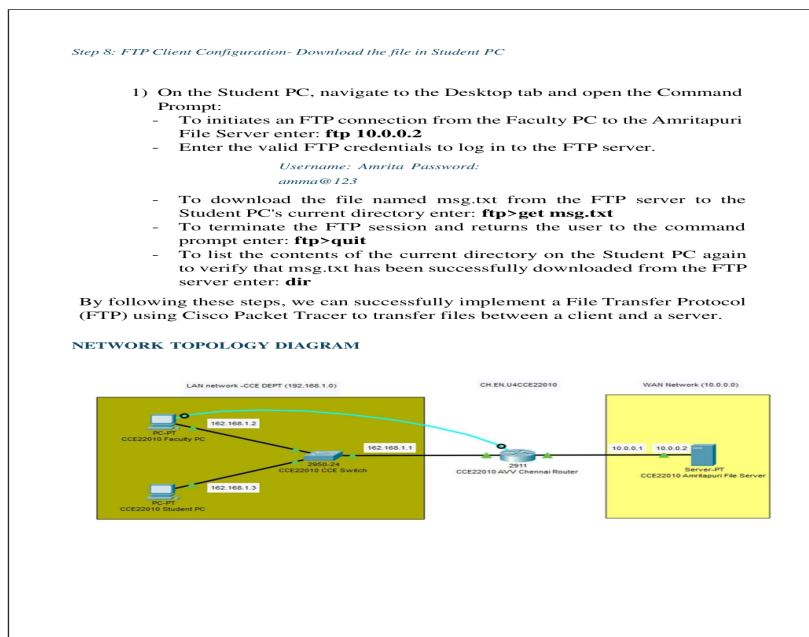


Figure: Network Topology

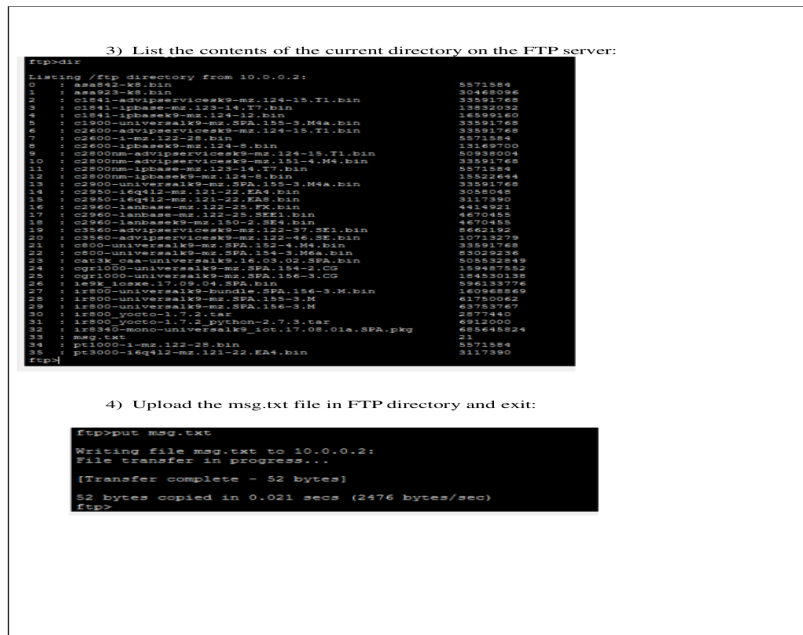


Figure: FTP Upload

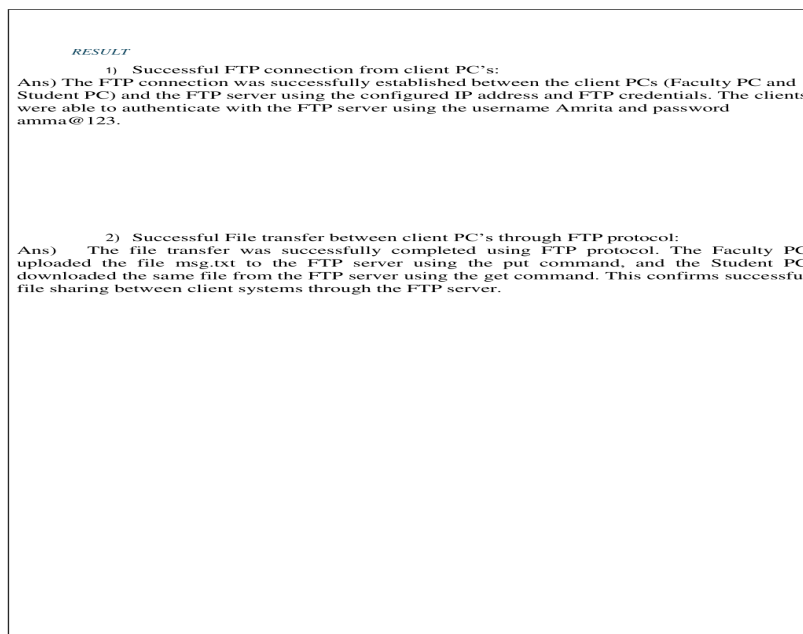


Figure: FTP Download

7. RESULTS & INFERENCE

Java File Transfer Results: The custom Java socket-based file transfer achieved complete success: • Server correctly identified sample.txt (1024 bytes / 1 KB) and initiated transfer • Client successfully received entire file content without data corruption • Bidirectional socket communication maintained stable throughout transfer duration • Proper resource cleanup prevented memory leaks and port binding issues

Cisco Packet Tracer FTP Results: • FTP server accepted connections from both client PCs simultaneously • Authentication system (username: Amrita, password: amma@123) functioned correctly • File upload (Faculty PC → Server) completed via put command • File download (Server → Student PC) completed via get command • Downloaded file integrity verified through local directory listing

Technical Inferences:

Inference 1 - Protocol Layering Benefits: Implementing FTP via Java sockets exposes lower-level protocol mechanics hidden by standard FTP clients. We directly manage TCP streams, buffer sizing, and connection state—knowledge valuable for debugging protocol issues or implementing custom protocols.

Inference 2 - Network Segmentation Necessity: Multi-segment topology (LAN/WAN) with router interconnection mirrors internet structure. Without routing between segments, Faculty and Student PCs couldn't communicate despite being on same physical network. This demonstrates why enterprises invest heavily in routing infrastructure.

Inference 3 - Authentication Security Considerations: Plain-text credential transmission (visible in terminal captures) represents significant security risk in production environments. Real deployments require TLS/SSL encryption (FTPS) or SSH tunneling to protect credentials and file contents from interception.

Inference 4 - Buffer Size Performance Impact: 4096-byte buffer balances efficiency (fewer syscalls) against memory usage (holding 4KB per iteration). For 1KB test file, single-read suffices. Large file transfers (GB-scale) benefit from larger buffers (8KB-64KB) reducing syscall overhead while managing memory constraints.

8. LAB SCENARIO SOLUTIONS

Part A Scenario: Secure Document Sharing - Java Implementation Q1: What file did you request from the server? Answer: The client requested "sample.txt" from the server. After server confirmed file existence (File.exists() returned true), the transfer process executed via socket streams. Total transferred volume was 1024 bytes (1 kilobyte), successfully persisted to local storage as "received_sample.txt". Q2: How does your client handle "file not found" situations? Answer: Upon receiving file request, server performs File.exists() validation check. If file absent:

- Server prints "File not found!" error message to console
- Server sends error notification to client via socket
- Client receives and displays error to user
- Client calls socket.close() to terminate connection cleanly

This prevents NullPointerException from attempting FileInputStream on non-existent file and provides user-friendly error reporting instead of stack trace.

Part B Scenario: Corporate Environment FTP Simulation Q3: What was the IP configuration? Answer:

- Faculty PC (Client 1): IPv4: 192.168.1.2, Subnet: 255.255.255.0, Gateway: 192.168.1.1
- Student PC (Client 2): IPv4: 192.168.1.3, Subnet: 255.255.255.0, Gateway: 192.168.1.1
- Amritapuri File Server: IPv4: 10.0.0.2, Subnet: 255.0.0.0, Gateway: 10.0.0.1

Q4: Demonstrate successful upload and download? Answer: Evidence provided via terminal screenshots showing:

- Upload: Faculty PC terminal shows "ftp>put msg.txt" with "226 Sending data" progress and "226 Transfer complete" confirmation
- Download: Student PC terminal shows "ftp>get msg.txt" with "1 Download successful" message
- Verification: Student PC dir command shows msg.txt present with correct timestamp/size

9. VIVA QUESTIONS & TECHNICAL DISCUSSION

Q1: How does file transfer happen using sockets? File transfer via sockets follows request-response pattern: Server opens ServerSocket on known port, client creates Socket and connects to server:port. Upon connection acceptance, server opens FileInputStream on target file, reads content into byte[], transmits via Socket.getOutputStream() using write(buffer, offset, length). Client receives via Socket.getInputStream(), writes to FileOutputStream, creating local copy. TCP guarantees ordered, reliable delivery of every byte. **Q2: What happens if file doesn't exist on server?** Server's File.exists() check returns false, triggering error branch: prints "File not found!", sends error to client, closes connection without attempting read operation. Client receives error message, displays to user, exits gracefully. Prevents crash from attempting FileInputStream on null/non-existent file object. **Q3: How do you simulate this in Packet Tracer?** Assemble network topology with Server-PT, Router 2911, Switch 2950-24, two PC-PTs. Configure IPs per subnet (192.168.1.x/24 for LAN, 10.0.0.x/8 for WAN). Enable FTP service on Server-PT, create user with credentials. Connect clients via Command Prompt using ftp , authenticate, use put/get for file operations. **Q4: What are FTP limitations?**

- Security: Credentials and data transmit in plaintext (no encryption)
- Firewall traversal: Requires specific ports opened (20/21) plus data ports
- Passive mode complications: NAT traversal difficulties for data connections
- Efficiency: Separate control channel adds latency vs single-socket designs
- Scalability: Single connection per server instance limits concurrency
- Large file performance: No resume capability for interrupted transfers

10. KEY LEARNINGS & TAKEAWAYS

Technical Competencies Mastered: 1. **Java Network Programming:** ServerSocket, Socket, FileInputStream, FileOutputStream usage for TCP socket file transfer implementation 2. **Protocol Understanding:** FTP command structure (USER, PASS, PUT, GET, QUIT), control/data channel separation, active/passive transfer modes 3. **Network Design:** Subnetting, routing between network segments, IP address planning, gateway configuration 4. **Tool Proficiency:** Cisco Packet Tracer device configuration, service setup, CLI-based FTP client usage 5. **Error Handling:** Defensive programming for file I/O, connection lifecycle management, user-friendly error reporting

Professional Skills Developed:

- **Documentation:** Writing structured lab reports with code samples, screenshots, and inference sections
- **Troubleshooting:** Diagnosing connectivity issues (ping, traceroute, port checks), identifying misconfigurations
- **Security Awareness:** Recognizing plaintext credential risks, understanding need for encrypted alternatives (SFTP/FTPS)
- **System Design:** Planning network topologies considering scalability, security zones, redundancy requirements

Foundation for Advanced Topics:

- Asynchronous I/O (NIO) for high-concurrency file servers
- TLS/SSL integration for secure file transfer
- Proxy/FTP configurations for enterprise environments
- Monitoring and logging for file transfer auditing
- Cloud storage integration (S3, Azure Blob, Google Cloud Storage)